Optimizing Latency in Microservices Architectures via a Distributed Cache Based on Consistent Hashing

Abdou Wahidi BELLO[1], Abdoul Kamal ASSOUMA[2], Tahirou DJARA[3]
and Emmanuel Godwin HOUENOU[4]

[1]Faculté des Sciences et Techniques, Université d'Abomey-Calavi, Abomey-Calavi, Benin
[2.3.4]École Polytechnique d'Abomey-Calavi, Université d'Abomey-Calavi, Abomey-Calavi, Benin
solfath@yahoo.fr, abdoulk1@hotmail.com, csm.djara@gmail.com, godwinmanu3@gmail.com

Abstract: In microservice architectures, reducing latency, managing distributed caching, and optimizing overall performance are key challenges. This article presents a distributed caching solution with automatic invalidation and consistent hashing to improve latency in microservices. Our approach uses Redis instances organized by Docker Compose, ensuring high availability without a single point of failure. Cache invalidation combines a TTL based strategy with event-based invalidation via Kafka, ensuring real-time data consistency. The implementation of consistent hashing enhances key distribution and minimizes reallocations topology changes. Experimental results demonstrate a high reduction in both latency and database overload.

Keywords: Distributed cache, Microservices, Consistent Hashing, Cache invalidation, Distributed architecture, Latency

1. Introduction

With the quick evolution of web and mobile applications, microservices architectures have become increasingly used. By breaking an application into multiple independent services, they offer greater flexibility and improved scalability. However, this methodology has limitation. Indeed, this approach introduces new challenges, particularly regarding performance management, latency, and data consistency.

In high-traffic microservices architectures, each service needs to access various resources and interact with data that is often centralized. The high volume of requests can quickly create bottlenecks, slowing down the entire application and degrading the user experience. To mitigate this, a distributed caching system is essential. By temporarily storing frequently accessed data in memory, it reduces access latency and eases the load on databases.

However, maintaining a high-performance and consistent cache in a distributed environment is a complex challenge. Cached data must remain synchronized with the database to ensure accuracy, raising concerns about data management and real-time updates. In this context, automatic cache invalidation emerges as a key solution. It ensures that cached data is refreshed in response to changes in data sources, preventing inconsistencies without introducing unnecessary overhead. Additionally, effective cache distribution is crucial to prevent overloading specific nodes. This is where the Consistent Hashing algorithm plays a vital role, enabling even data distribution across cache nodes while minimizing disruptions when nodes are added or removed.

Our work focuses on designing and implementing a distributed cache management system that integrates automatic invalidation mechanisms and Consistent Hashing to ensure efficient cache distribution and optimized performance in microservices architectures.

2. Previous work

A. Review of existing literature

Microservice architectures have profoundly transformed the way distributed applications are designed and deployed, introducing new challenges in terms of data management and performance. Li et al. [1] demonstrated that distributed caching has become one of the most critical components for maintaining performance in these complex architectures. The work of Gao et al. [2] showed significant reductions in response times, up to 35%, through the appropriate

Received: May 6th, 2024. Accepted: September 4th, 2025
DOI: 10.15676/ijeei.2025.17.3.2

implementation of distributed caching systems. This observation was reinforced by Seth et al. [3], who analyzed the impact of caching on microservice scalability.

The evolution of caching systems has followed that of distributed architectures. Faustino et al. [4] documented this progression, from monolithic caching systems to modern distributed solutions. Their research particularly highlighted the growing importance of horizontal scalability in microservice environments. Yan et al. [5] complemented this analysis by studying the impact of containerization on caching strategies. Data consistency management in distributed caching systems has been the subject of extensive research. Shukur et al. [6] introduced a novel coherence protocol adapted to distributed architectures. Their approach was enhanced by Laigner et al. [7] with research on data synchronization techniques and optimized replication mechanisms. Cache invalidation in microservice architectures represents a key challenge in ensuring data freshness while minimizing system performance impact. Several approaches have been explored in the literature, each presenting advantages and limitations depending on the application context. Early invalidation strategies primarily relied on Time-To-Live (TTL), a straightforward approach where each cache entry has a predetermined lifetime after which it is automatically removed [8]. However, this method can lead to inconsistencies if data changes occur before cached entries expire. To address this issue, Liu et al. [9] developed adaptive algorithms that dynamically adjust cache lifetimes according to data access patterns.

A more sophisticated alternative is event-based invalidation, which relies on propagating update notifications when underlying data is modified. Sourlas et al. [10] introduced a publish/subscribe framework enabling efficient real-time cache synchronization. This model was extended by Crudu and MoldStud Research Team [11], who investigated event propagation in large-scale distributed systems and demonstrated significant reductions in update latency.

One of the primary limitations of event-based approaches is the network overhead generated by notification distribution. de Araujo [12] proposed event compression and aggregation techniques to minimize bandwidth utilization. Additionally, Alachisoft offered an alternative solution with NCache, which provides automatic synchronization with underlying databases [13]. This technology eliminates the need for manual implementation of invalidation mechanisms but remains limited to specific databases and proprietary technologies.

Latency reduction remains a paramount objective in microservice architectures. Sobri et al. [14] established a direct correlation between caching strategies and overall system latency. Zhang et al. [15] proposed an innovative cache management framework called MuCache that substantially reduces latency. Alam et al. [16] introduced criteria for optimal distributed cache implementation to achieve better latency reduction rates. Similarly, Anju Bhole [17] analyzed and proposed a hybrid caching strategy to enhance microservice performance.

B. Critical analysis of existing solutions

A review of the literature reveals several important limitations in current approaches to improved distributed cache management in microservice architectures. One recurring criticism is that many solutions focus on isolated aspects of the problem, without proposing an overall framework or optimal solution. For example, Newman [18] points out that existing approaches tend to focus on specific challenges, such as distributed data management or inter-service communication, without holistically addressing the complex interactions between these different aspects. This fragmentation was also highlighted by Ramu [19], who analyzed the impact of partial integration of solutions on the overall performance of microservices architectures. Their study shows that the adoption of point solutions, without coordination between them, can lead to systemic inefficiencies, such as communication bottlenecks or data consistency problems. For example, the use of isolated caching mechanisms without synchronization with data management systems can lead to inconsistencies and performance degradation.

A major limitation in the field of distributed cache management is the absence of a comprehensive methodology integrating the various technical aspects, such as consistent hashing, distributed cache implementation and invalidation strategies. Although these techniques have been widely studied separately, few works propose a unified approach to combining them

optimally, particularly in the context of microservice architectures. For example, Nishtala et al. [20], in their work on large-scale cache systems at Facebook, have shown that isolated use of consistent hashing or specific invalidation strategies can improve performance, but that poorly coordinated integration of these techniques can lead to inefficiencies. They emphasize the need for a holistic approach to optimizing latency and data consistency in distributed environments. Furthermore, Karger et al [21], in their seminal paper on consistent hashing, highlighted the benefits of this technique for load balancing, but did not explore its integration with invalidation or cache management mechanisms in modern architectures such as microservices.

Although event-based and TTL invalidation mechanisms offer distinct advantages, the literature on hybrid approaches remains scarce. Our work aims to fill this gap by proposing a concrete methodology illustrated by an implementation combining consistent hashing with a hybrid (event/TTL) invalidation strategy. The main aim of this approach is to significantly reduce latency in microservice architectures, an important objective identified by Sobri [14] and Zhang [15]. Our solution thus addresses an essential need to improve the overall performance of modern distributed applications by directly tackling this latency challenge.

3. Methodology

A. System architecture design

The architecture of our solution is illustrated in the figure below. It is based on a microservices approach integrating a distributed cache to optimize latency and improve system performance. This architecture is composed of several interconnected blocks, each playing a specific role in the overall operation of the system.

Here's a presentation of this architecture through the different blocks and phases numbered on Fig. 1 :

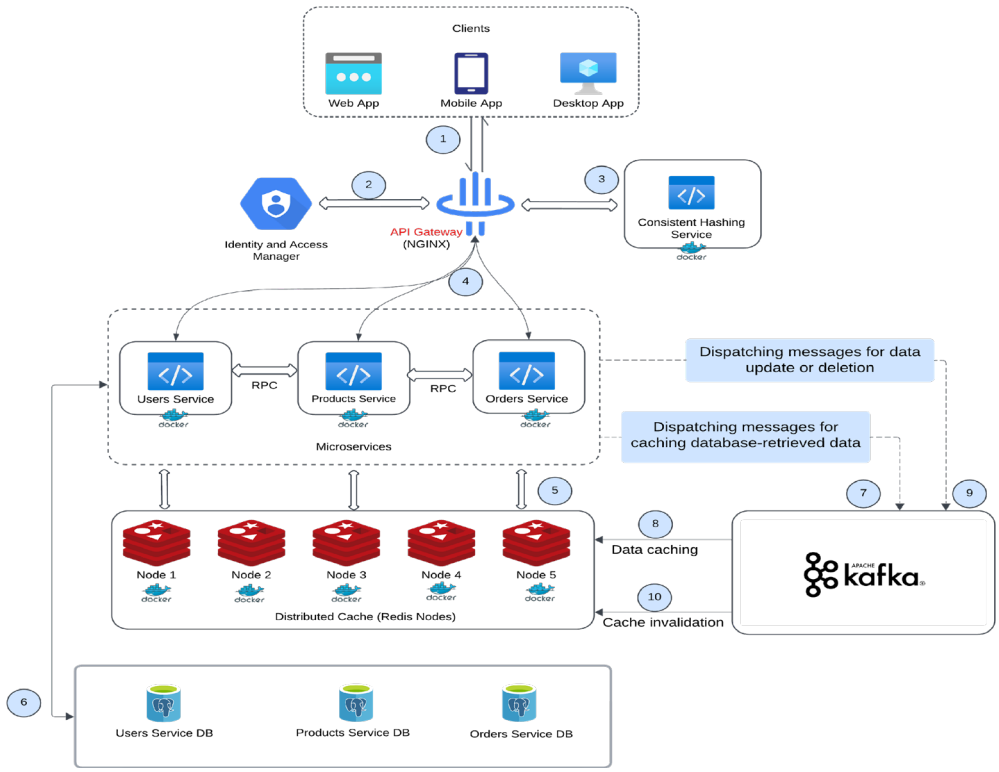


Figure 1. System architecture

1. Client interaction with the system: Clients (web, mobile, desktop applications) send requests to the API Gateway (NGINX). This gateway centralizes all incoming requests and acts as a single point of entry to the system's various services.
2. Authentication and access management: before routing a request to a microservice, the API Gateway checks the user's authentication and authorization by querying the identity and access manager. This mechanism ensures that only valid requests are processed.
3. Determining the cache node with the Consistent Hashing service: Once access has been validated, the request URL is passed on to the Consistent Hashing service. This service, which operates independently of microservices, determines on which Redis node the data will be or is stored and returns it to the API Gateway. The API Gateway then rewrites the URL, integrating the determined cache node and redirecting the request to the appropriate service.
4. Processing of requests by microservices: After the request has been routed by the API Gateway, the selected microservice processes the request internally. The implementation of this processing may require calling other microservices for specific reasons; in such cases, inter-service communication is carried out via RPC calls to limit coupling and facilitate system scalability.
5. Retrieving data from the distributed cache : When a microservice receives a request, it first queries the distributed cache to check whether the requested data is already stored (cache hit). If the data is found, it is immediately returned to the client, reducing latency and avoiding a database query.
6. Data retrieval from databases: If the data is not found in the cache (cache miss), the microservice queries its database to retrieve it.
7. Send message to kafka for asynchronous data caching: Once obtained, the data is sent back to the client. The relevant microservice sends a message to the kafka caching service.
8. Caching of previously retrieved data: The kafka caching service receives the message and stores it in the distributed cache for rapid response.
9. Sending messages to kafka following data update or delete events: When data is updated or deleted in a database, the microservice concerned sends an invalidation message to kafka.
10. Distributed cache invalidation: The message is used to invalidate the cache. This approach guarantees the consistency of information between the cache and the databases, thus avoiding the use of obsolete data.

B. Microservices development and containerization

To test and validate our approach, we opted for an e-commerce application backend made up of three main microservices:

- Users Service: manages user information, including authentication, account creation and profile management;
- Products Service: manages the product catalog, allowing products to be added, deleted or updated;
- Orders Service: manages user orders.

Each service exposes RESTful API endpoints to interact with other system components. Microservices have been developed in Django following proven architectural patterns such as the API Gateway pattern and the Database per service pattern, ensuring modularity, scalability and maintainability.

The services have been dockerized and orchestrated with Docker Compose via a docker-compose.yml file defining the configuration for the deployment of all components. Here's the list of configured services:

- hashing-service: Hash service for data distribution
- nginx: Web server used as an API gateway
- redis-node-1 to redis-node-5: Redis instances for distributed caching
- zookeeper: Zookeeper server for Kafka metadata management

- kafka: Kafka broker for communication between services
- kafka-ui: User interface for Kafka monitoring
- users-service: User management service
- products-service: Product management service
- orders-service: Order management service
- cache-writer: service for writing data to the cache
- cache-invalidator: service for invalidating cache data.

C. Development of the Consistent Hashing service

The Consistent Hashing service is the core element of our architecture, enabling efficient and balanced distribution of data in the distributed cache system. This service determines precisely which cache node is responsible for each request.

a) The imbalance problem and the virtual node solution

With k physical nodes randomly distributed around the ring, statistical theory shows that ring segments (and therefore the corresponding load) exhibit significant variability. In an ideal distribution, each node should manage exactly 1/k of the total space. In practice, however, the distribution can be very uneven.

To quantify this imbalance, we use the coefficient of variation (CV), which is the ratio between the standard deviation and the mean of the loads per node:

$$CV = \frac{\sigma}{\mu} \tag{1}$$

A high coefficient of variation indicates a strong disparity in key distribution. Intuitively, a CV of X% means that a node can receive up to ±X% more or less load than the average. With a random placement of k physical nodes, statistical theory tells us that this coefficient of variation is approximately:

$$CV = \frac{1}{\sqrt{k}} \tag{2}$$

For our system with k = 5 physical nodes, we would have a theoretical coefficient of variation of 44.7%, which represents significant deviations in key distribution. To solve this problem, we implemented the virtual node solution proposed by DeCandia et al. (2007).

Each physical node is represented by v=200 virtual points on the ring, pseudo-randomly distributed. This approach reduces the theoretical coefficient of variation to around 3.2%, ensuring a sufficiently balanced distribution while maintaining a reasonable memory footprint.

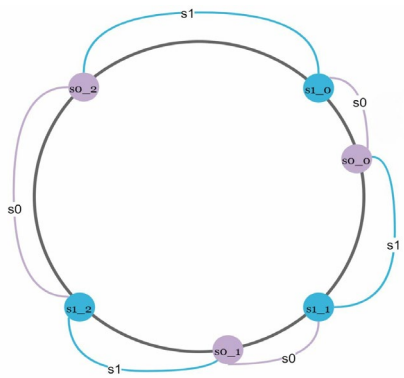


Figure 2. Illustration of hash ring with virtual nodes

b) Technical implementation

We chose the SortedDict method from the sortedcontainers library to represent the virtual ring. This structure offers guaranteed O(log n) performance for search, insertion, and delete operations, with no risk of degeneracy, unlike standard BST (Binary Search Tree). The hash function XXH32 was selected for its execution speed (9.7 GB/s) and distribution quality, outperforming Murmur3, SHA1, and MD5 in our usage context.

The Singleton pattern has been implemented via a metaclass to guarantee ring consistency across all requests. Dynamic node addition and removal are handled by methods that maintain consistency between physical and virtual nodes.

The algorithm determines the node responsible for each key by calculating its hash value and searching for the next position in the ring whith bisect_right method of SortedDict library with complexity O(log n).

The service is exposed via a REST API with an endpoint */compute_node_key*, which is called by the API Gateway to determine the Redis node that stores or will store the data linked to each request. This node (the identifier) is then added to the request URL as a parameter for processing by the various microservices.

D. Development of distributed cache management package

To facilitate integration and avoid code duplication, we have developed a Python package encapsulating the distributed cache management logic, consisting of three main modules:

- Cache middleware (middleware.py) : The middleware CacheMiddleware intercepts HTTP GET requests to check whether the requested data is available in the cache. It verifies the presence of a cache node specified in the request URL parameters, uses the CacheManager class to generate a key based on the request URL and returns cache data directly if available, otherwise transfers the request for normal processing.
- Cache manager (cache_manager.py) : This module contains the definition of the CacheManager class, which encapsulates all cache management logic. It manages: connection to the specified Redis instance, generation of URL-based cache keys, data retrieval and storage with configurable TTL, and management of associations between entities and keys via Redis sets.
- Asynchronous cache writing (cache_writer.py) : This module defines a service that listens to a Kafka data flow (cache_writing) and writes data to the cache according to the messages received. It provides the following functions: optimized storage of data from Kafka messages and guaranteed rapid availability of recent data without intervention by microservices.

When a resource is modified (updated or deleted), it is essential to invalidate the associated cached data to avoid returning obsolete information to clients. To achieve this, we have designed a system based on Apache Kafka to notify modifications, and a dedicated cache-invalidator service that constantly listens to Kafka notifications (messages) to process them and invalidate the cache.

a) Kafka integration

Each microservice sends messages during update or delete operation. These messages contain the operation (UPDATE/DELETE) of entity concerned and the identifier.

b) Cache invalidation mechanism

The cache-invalidator service listens to Kafka messages on the cache_invalidation data flow. It retrieves all the keys associated with the resource via a Redis set. For updates, it refreshes the data in the cache, and for delete, it removes all associated entries.

This approach is complemented by TTL-based invalidation, ensuring that data is automatically deleted after a specified time period. This dual strategy (proactive invalidation and TTL) guarantees robust cache management, avoiding obsolete data and the accumulation of unnecessary entries.

4. Results and discussion

A. Results

1) Cache Data Distribution with Consistent Hashing

a) Data Distribution Analysis

To evaluate the efficiency of data distribution using consistent hashing, we implemented a script that collects statistics on the distribution of keys across Redis nodes. These statistics were extracted at strategic points during our tests, particularly at the 4th and 7th minute of an 8 minute test.

- *At 4th Minute*

  This point marks a stability phase in the load test: the first keys inserted into the cache are still present (their TTL has not yet expired), and no new data is being added to the system at this stage. The cache thus contains a stable volume of data, allowing for a more accurate evaluation of key distribution across Redis nodes before successive expirations affect the observed balance. Figure 3 presents the script's output showing the number of cache keys on each node at the 4th minute.

```
→  infrastructure git:(dev) ✗ ./nodes_stats.sh
Nombre de clés sur redis-node-1 :
224
----------------------------
Nombre de clés sur redis-node-2 :
224
----------------------------
Nombre de clés sur redis-node-3 :
251
----------------------------
Nombre de clés sur redis-node-4 :
242
----------------------------
Nombre de clés sur redis-node-5 :
211
----------------------------
→  infrastructure git:(dev) ✗
```

Figure 3. Data distribution across nodes at the 4th minute

- *At 7th Minute*

  At this point, data stored at the beginning of the test, especially those inserted around the 2nd minute, have expired, having reached their time-to-live. Meanwhile, new data have been inserted into the cache throughout the test. This phase allows observation of the redistribution of keys after initial expirations. Figure 4 shows the number of cache keys on each node at the 7th minute.

```
→ infrastructure git:(dev) ✗ ./nodes_stats.sh
Nombre de clés sur redis-node-1 :
560
----------------------------
Nombre de clés sur redis-node-2 :
523
----------------------------
Nombre de clés sur redis-node-3 :
581
----------------------------
Nombre de clés sur redis-node-4 :
526
----------------------------
Nombre de clés sur redis-node-5 :
514
----------------------------
```

Figure 4. Data distribution across nodes at the 7th minute

Figure 5 below illustrates the number of keys distributed across the five Redis nodes at these two points:

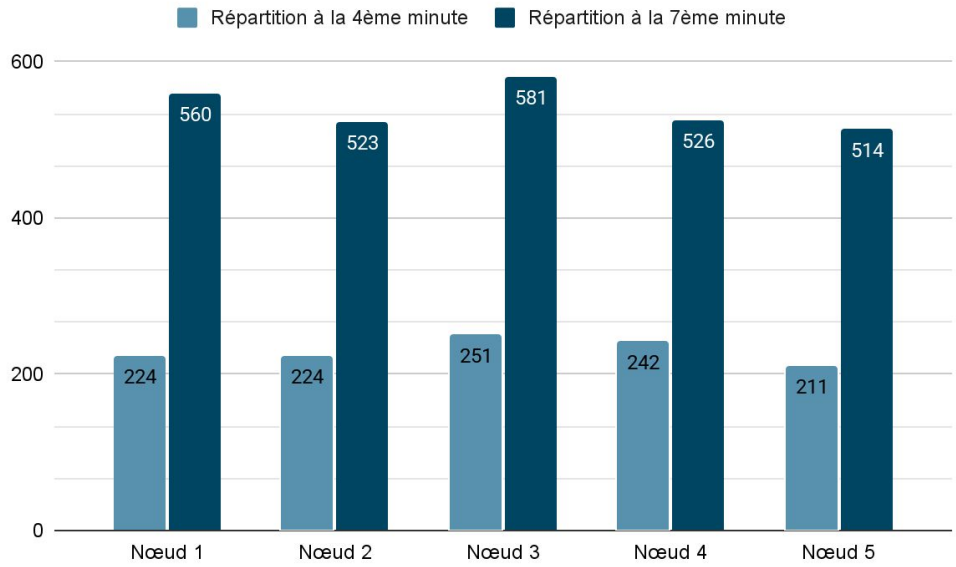


Figure 5. Distribution of cached data using consistent hashing

We observe that the distribution of keys remains relatively balanced across nodes, despite the successive expirations and continuous data insertion. To support this observation, we computed the correlation coefficient for the number of keys stored on each node at both time points:

Table 1. Correlation coefficient of key distribution at two measurement points

	Mean (μ)	Standard Deviation (σ)	Coefficient of Variation (CV)
4th minute	230,4 keys	14,26 keys	6,19 %
7th minute	540,8 keys	25,43 keys	4,70 %

These values indicate low variability between nodes, especially at 7th minute where the CV falls below the commonly accepted 5% threshold. At 4th minute, although the CV slightly exceeds this threshold, the observed dispersion remains moderate and acceptable. The objective of fair key distribution is achieved, demonstrating that our implementation of consistent hashing ensures an effective load balancing across cache nodes.

b). Adding a New Node to the Ring

To test the system's ability to dynamically handle the addition of new nodes, we added a new Redis service (redis-node-6) by modifying the docker-compose.yml configuration file. We then used Postman to send a request to add the new node to the hash ring. Figure 6 illustrates this step in Postman.

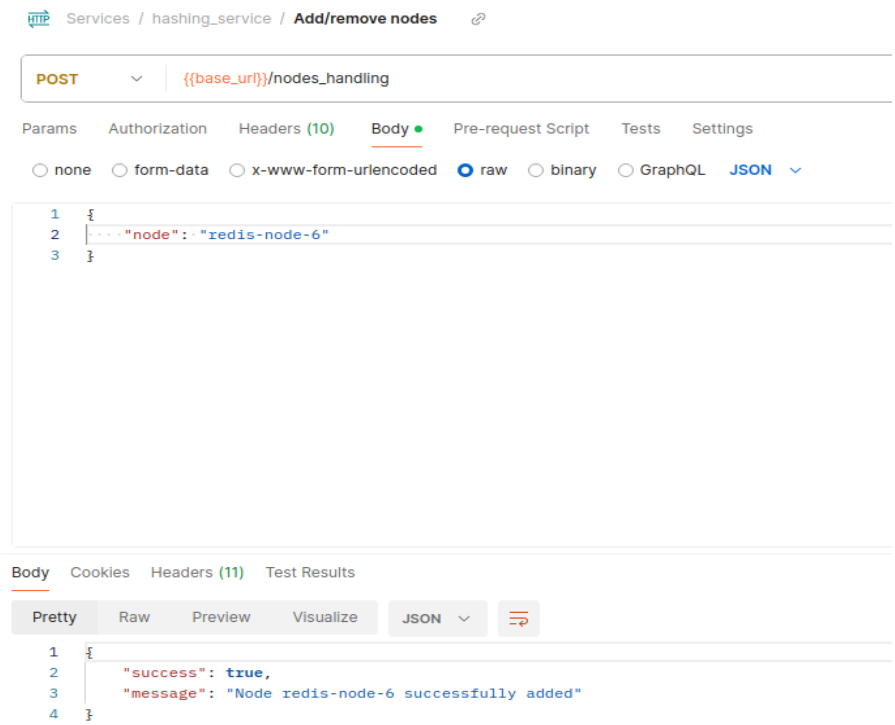


Figure 6. Adding a new node via a Postman request

After adding the node, we observed in RedisInsight that data began to be distributed to the new node. Figure 7 shows the data state on this node.

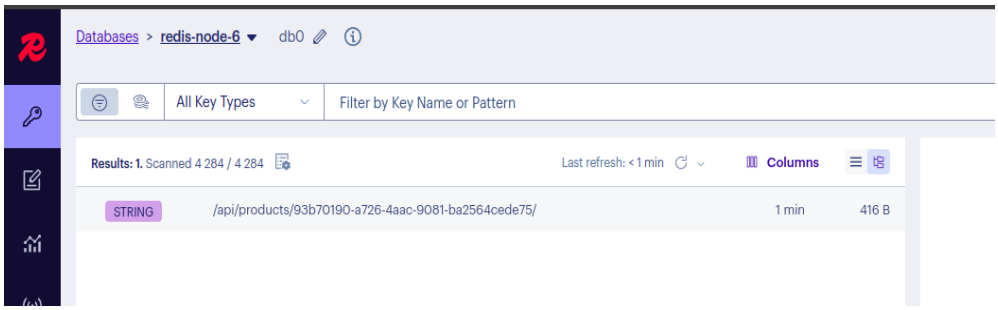


Figure 7. Data distribution after adding the new node

c) Node Removal to Simulate a Failure

To simulate a failure or unavailability of a node, we removed redis-node-6 from the ring. In line with the consistent hashing principle, data previously assigned to the removed node (redis-node-6) were reassigned on subsequent requests to the next node in the ring, namely redis-node-1, since redis-node-6 was the last node in the hash circle. Figure 8 illustrates this reassignment.

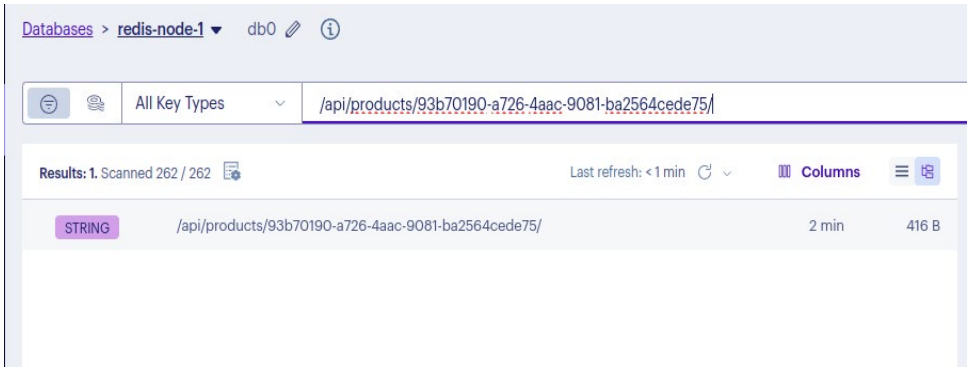


Figure 8. Data reassignment after node removal

2) Load Testing Results with Locust

To evaluate the effectiveness of our distributed cache management system, we performed a series of load tests using the Locust tool. Tests were conducted under two different configurations: with and without our proposed caching solution. This approach allows us to objectively quantify the impact of our implementation on the system's overall performance.

We designed five test scenarios with an increasing number of concurrent users:

- 10 users (spawn rate: 1 user/second)
- 100 users (spawn rate: 1 user/second)
- 250 users (spawn rate: 2 users/second)
- 500 users (spawn rate: 5 users/second)
- 1000 users (spawn rate: 10 users/second)

Each scenario was executed twice: once with our cache solution and once without it, enabling direct performance comparison under identical conditions.

a) Results for 10 Users

In this scenario with 10 simulated users, we performed:

- One test with our distributed caching solution
- One test without our caching solution

Test with Our Cache Solution

Figure 9 presents request statistics for the test with 10 simulated users using our distributed caching solution. These statistics show that the system handled a total of 2032 requests without any failures, with an average response time of 18.13 ms.

Type	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s
	Aggregated	2032	0	18.13	6	464	14543.67	3.98	0

Figure 9. Request statistics for 10 users with cache solution

Figure 10 illustrates the response time distribution for the same test. The response time analysis reveals that 50% of requests were handled in under 9 ms, while 95% were completed in under 34 ms, indicating excellent system responsiveness. At the 99th percentile, response times remained under 280 ms. The maximum response time observed was 460 ms.

Method	Name	50%ile (ms)	60%ile (ms)	70%ile (ms)	80%ile (ms)	90%ile (ms)	95%ile (ms)	99%ile (ms)	100%ile (ms)
	Aggregated	9	10	10	12	29	34	280	460

Figure 10. Response time statistics for 10 users with cache solution

Test without Our Cache Solution

Figure 11 presents request statistics for the test with 10 simulated users without using our distributed caching solution. These statistics show that the system handled a total of 1990 requests without any failures, with an average response time of 33.36 ms.

Type	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s
	Aggregated	1990	0	33.36	16	294	15474.16	3.58	0

Figure 11. Request statistics for 10 users without cache solution

Figure 12 displays response time statistics for the test without the caching solution. Analysis reveals that 50% of requests were handled in under 26 ms, while 95% were completed in under 73 ms. At the 99th percentile, response times reached 200 ms. The maximum response time observed was 290 ms.

Method	Name	50%ile (ms)	60%ile (ms)	70%ile (ms)	80%ile (ms)	90%ile (ms)	95%ile (ms)	99%ile (ms)	100%ile (ms)
	Aggregated	26	27	27	29	52	73	200	290

Figure 12. Response time statistics for 10 users without cache solution

b) Results for 100 Users

For this scenario with 100 simulated users, we performed:

- One test with our distributed caching solution
- One test without our caching solution

Test with Our Cache Solution

Figure 13 presents request statistics for the test with 100 simulated users using our distributed caching solution. These statistics show that the system handled a total of 18410 requests without any failures, with an average response time of 24.06 ms and a throughput of 35.54 requests per second.

Type	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s
	Aggregated	18410	0	24.06	5	2280	14013.51	35.54	0

Figure 13. Request statistics for 100 users with cache solution

Figure 14 illustrates the response time distribution for the same test. The response time analysis reveals that 50% of requests were handled in under 10 ms. At the 95th percentile, response times remained under 39 ms, indicating good stability even under increased load. At the 99th percentile, response times reached 340 ms. The maximum response time observed was 2300 ms, primarily for authentication requests (POST /token) and some specific GET requests.

Method	Name	50%ile (ms)	60%ile (ms)	70%ile (ms)	80%ile (ms)	90%ile (ms)	95%ile (ms)	99%ile (ms)	100%ile (ms)
POST	http://172.17.0.1:8080/realms/eng-thesis-project/protocol/openid-connect/token	36	37	39	41	45	50	2100	2100
GET	/order-items/	21	22	22	23	24	25	91	2300
GET	/products/d2141259-e75f-416a-86f0-a22e5ffcd4de/	9	9	10	10	11	12	99	2300
	Aggregated	10	10	10	12	30	39	340	2300

Figure 14. Response time statistics for 100 users with cache solution

Test without Our Cache Solution

Figure 15 presents request statistics for the test with 100 simulated users without using our distributed caching solution. These statistics show that the system handled a total of 20862 requests without any failures, with an average response time of 32.83 ms and a throughput of 33.43 requests per second.

Type	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s
	Aggregated	20862	0	32.83	14	708	14236.19	33.43	0

Figure 15. Request statistics for 100 users without cache solution

Figure 16 displays response time statistics for the test without the caching solution. Analysis reveals that 50% of requests were handled in under 28 ms, while 95% were completed in under 77 ms, almost double the times observed with our caching solution under the same load. At the 99th percentile, response times remained under 140 ms. The maximum response time observed was 710 ms.

Method	Name	50%ile (ms)	60%ile (ms)	70%ile (ms)	80%ile (ms)	90%ile (ms)	95%ile (ms)	99%ile (ms)	100%ile (ms)
	Aggregated	28	28	29	30	36	77	140	710

Figure 16. Response time statistics for 100 users without cache solution

c) Results for 250 Users

For this scenario with 250 simulated users, we performed:

- One test with our distributed caching solution
- One test without our caching solution

Test with Our Cache Solution

Figure 17 presents request statistics for the test with 250 simulated users using our distributed caching solution. These statistics show that the system handled a total of 43,184 requests with 1 failure, an average response time of 27.46 ms, and a throughput of 84.79 requests per second.

Type	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s
	Aggregated	43184	1	27.46	5	1518	13870.02	84.79	0

Figure 17. Request statistics for 250 users with cache solution

Figure 18 illustrates the response time distribution for the same test. Analysis reveals that 50% of requests were handled in under 10 ms. At the 95th percentile, response times remained under 36 ms, indicating good stability even under increased load. At the 99th percentile, response times reached 520 ms. The maximum response time observed was 1,500 ms, primarily for PATCH and POST requests.

Method	Name	50%ile (ms)	60%ile (ms)	70%ile (ms)	80%ile (ms)	90%ile (ms)	95%ile (ms)	99%ile (ms)	100%ile (ms)
PATCH	/products/bc92d3be-ee7e-4b7d-a898-630537c37be6/	180	230	290	370	460	510	590	720
GET	/users/	9	10	10	10	11	12	21	980
POST	/order-items/	950	1000	1100	1200	1300	1400	1500	1500
	Aggregated	10	10	10	11	23	36	520	1500

Figure 18. Response time statistics for 250 users with cache solution

Test without Our Cache Solution

Figure 19 presents request statistics for the test with 250 simulated users without using our distributed caching solution. These statistics show that the system handled a total of 46,355 requests without any failures, with an average response time of 41.18 ms, and a throughput of 86.47 requests per second.

Type	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s
	Aggregated	46355	0	41.18	15	532	14180.06	86.47	0

Figure 19. Request statistics for 250 users without cache solution

Figure 20 displays response time statistics for the test without the caching solution. Analysis reveals that 50% of requests were handled in under 29 ms. At the 95th percentile, response times reached 110 ms, almost triple the times observed with our caching solution under the same load. At the 99th percentile, response times remained under 280 ms. The maximum response time observed was 530 ms.

Method	Name	50%ile (ms)	60%ile (ms)	70%ile (ms)	80%ile (ms)	90%ile (ms)	95%ile (ms)	99%ile (ms)	100%ile (ms)
	Aggregated	29	29	31	34	78	110	280	530

Figure 20. Response time statistics for 250 users without cache solution

d) Results for 500 Users

For this scenario with 500 simulated users, we performed:

- One test with our distributed caching solution
- One test without our caching solution

Test with Our Cache Solution

Figure 21 presents request statistics for the test with 500 simulated users using our distributed caching solution. These statistics show that the system handled a total of 86,182 requests without any failures, with an average response time of 53.69 ms, and a throughput of 171.38 requests per second.

Type	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s
	Aggregated	86182	0	53.69	5	2754	13977.54	171.38	0

Figure 21. Request statistics for 500 users with cache solution

Figure 22 illustrates the response time distribution for the same test. Analysis reveals that 50% of requests were handled in under 10 ms. At the 95th percentile, response times remained under 36 ms, indicating good stability even under increased load. At the 99th percentile, response times reached 1,700 ms. The maximum response time observed was 2,800 ms, primarily for PATCH and POST requests.

Method	Name	50%ile (ms)	60%ile (ms)	70%ile (ms)	80%ile (ms)	90%ile (ms)	95%ile (ms)	99%ile (ms)	100%ile (ms)
PATCH	/products/93b70190-a726-4aac-9081-ba2564cede75/	930	1000	1100	1300	1500	1700	1700	1800
PATCH	/products/bc92d3be-ee7e-4b7d-a898-630537c37be6/	900	990	1100	1300	1500	1600	1700	1800
PATCH	/products/d2141259-e75f-416a-86f0-a22e5ffcd4de/	970	1100	1200	1400	1600	1700	1800	1800
PATCH	/products/11fae433-3d97-404a-894f-df34b30b95c6/	890	990	1100	1300	1600	1700	1800	2000
POST	/order-items/	2000	2000	2100	2200	2400	2500	2700	2800
	Aggregated	10	10	11	12	23	36	1700	2800

Figure 22. Response time statistics for 500 users with cache solution

Test without Our Cache Solution

Figure 23 presents request statistics for the test with 500 simulated users without using our distributed caching solution. These statistics show that the system handled a total of 82,010 requests with 1 failure, an average response time of 169.93 ms, and a throughput of 163.61 requests per second.

Type	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s
	Aggregated	82010	1	169.93	16	2502	13932.39	163.61	0

Figure 23. Request statistics for 500 users without cache solution

Figure 24 displays response time statistics for the test without the caching solution. Analysis reveals that 50% of requests were handled in under 67 ms. At the 95th percentile, response times reached 630 ms, almost double the times observed with our caching solution under the same load. At the 99th percentile, response times remained under 1,700 ms. The maximum response time observed was 2,500 ms.

Method	Name	50%ile (ms)	60%ile (ms)	70%ile (ms)	80%ile (ms)	90%ile (ms)	95%ile (ms)	99%ile (ms)	100%ile (ms)
PATCH	/products/93b70190-a726-4aac-9081-ba2564cede75/	1300	1500	1600	1900	2100	2300	2400	2400
PATCH	/products/bc92d3be-ee7e-4b7d-a898-630537c37be6/	1300	1400	1500	1800	2000	2200	2400	2400
PATCH	/products/d2141259-e75f-416a-86f0-a22e5ffcd4de/	1300	1500	1600	1900	2000	2300	2400	2500
	Aggregated	67	93	130	210	390	630	1700	2500

Figure 24. Response time statistics for 500 users without cache solution

e) Results for 1000 Users

For this scenario with 1000 simulated users, we performed:

- One test with our distributed caching solution
- One test without our caching solution

Test with Our Cache Solution

Figure 25 presents request statistics for the test with 1000 simulated users using our distributed caching solution. These statistics show that the system handled a total of 177,493 requests with 2 failures, an average response time of 156.16 ms, and a throughput of 332.29 requests per second.

Type	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s
	Aggregated	177493	2	156.16	5	7201	13993.42	332.29	0

Figure 25. Request statistics for 1000 users with cache solution

Figure 26 illustrates the response time distribution for the same test. Analysis reveals that 50% of requests were handled in under 17 ms. At the 95th percentile, response times remained under 120 ms, indicating good stability even under increased load. At the 99th percentile, response times reached 5,100 ms. The maximum response time observed was 7,200 ms, primarily for PATCH requests.

Method	Name	50%ile (ms)	60%ile (ms)	70%ile (ms)	80%ile (ms)	90%ile (ms)	95%ile (ms)	99%ile (ms)	100%ile (ms)
GET	/products/	16	18	21	27	38	53	130	4700
GET	/products/bc92d3be-ee7e-4b7d-a898-630537c37be6/	16	18	21	26	38	53	120	5000
PATCH	/products/11fae433-3d97-404a-894f-df34b30b95c6/	4000	4700	5400	6000	6600	6700	6900	6900
PATCH	/products/bc92d3be-ee7e-4b7d-a898-630537c37be6/	4200	4900	5500	6000	6600	6700	6800	6900
PATCH	/products/d2141259-e75f-416a-86f0-a22e5ffcd4de/	4300	5000	5700	6100	6600	6700	6900	6900
PATCH	/products/93b70190-a726-4aac-9081-ba2564cede75/	4200	5000	5600	6100	6600	6700	6800	7200
	Aggregated	17	20	25	34	54	120	5100	7200

Figure 26. Response time statistics for 1000 users with cache solution

Test without Our Cache Solution

Figure 27 presents request statistics for the test with 1000 simulated users without using our distributed caching solution. These statistics show that the system handled a total of 124,927 requests with 4 failures, an average response time of 599.93 ms, and a throughput of 236.52 requests per second.

Type	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s
	Aggregated	124927	4	599.93	14	3138	14091.49	236.52	0.01

Figure 27. Request statistics for 1000 users without cache solution

Figure 28 displays response time statistics for the test without the caching solution. Analysis reveals that 50% of requests were handled in under 120 ms. At the 95th percentile, response times reached 2,400 ms, almost double the times observed with our caching solution under the same load. At the 99th percentile, response times remained under 2,700 ms. The maximum response time observed was 3,100 ms.

Method	Name	50%ile (ms)	60%ile (ms)	70%ile (ms)	80%ile (ms)	90%ile (ms)	95%ile (ms)	99%ile (ms)	100%ile (ms)
PATCH	/products/d2141259-e75f-416a-86f0-a22e5ffcd4de/	45	59	170	1200	2100	2400	2800	2900
GET	/products/	620	1100	1600	1900	2300	2600	2800	3000
GET	/products/93b70190-a726-4aac-9081-ba2564cede75/	680	1200	1600	1900	2300	2600	2800	3000
GET	/products/11fae433-3d97-404a-894f-df34b30b95c6/	630	1200	1600	1900	2200	2600	2800	3100
GET	/products/bc92d3be-ee7e-4b7d-a898-630537c37be6/	630	1200	1600	1900	2300	2600	2800	3100
GET	/products/d2141259-e75f-416a-86f0-a22e5ffcd4de/	690	1200	1600	1900	2300	2600	2800	3100
	Aggregated	120	220	630	1500	2000	2400	2700	3100

Figure 28. Response time statistics for 1000 users without cache solution

Load test results show that our distributed cache solution significantly improves system performance, particularly for response times and request throughput. For the majority of requests (up to the 95th percentile), response times remain below 1 second, even under a high load of 1,000 users. This demonstrates that our solution is effective in reducing latency and improving the user experience.

The table below shows a summary of performance for the five test scenarios (10, 100, 250, 500 and 1000 users). It compares response times at the 95th, 99th and 100th percentiles with and without our solution, as well as the rate of response time reduction.

Table 2. Comparison of response times with and without our solution

Number of users	Metric	Results		Reduction rate (%)
		Without our solution	With our solution	
10	Requests	1990	2032	-
	95%ile (ms)	73	34	-53,4
	99%ile (ms)	200	280	+40,0
	100%ile (ms)	290	460	+58,6
100	Requests	20862	18410	-
	95%ile (ms)	77	39	-49,4
	99%ile (ms)	140	340	+142,9
	100%ile (ms)	710	2300	+223,9
250	Requests	46355	43184	-
	95%ile (ms)	110	36	-67,3
	99%ile (ms)	280	520	+85,7
	100%ile (ms)	530	1500	+183,0
500	Requests	82010	86182	-
	95%ile (ms)	630	36	-94,3
	99%ile (ms)	1700	1700	0,0
	100%ile (ms)	2500	2800	+12,0
1000	Requests	124927	177493	-
	95%ile (ms)	2400	120	-95,0
	99%ile (ms)	2700	5100	+88,9
	100%ile (ms)	3100	7200	+132,3

This table shows that our solution significantly reduces response times up to the 95th percentile, with gains ranging from 49.35% for 100 users to 95.00% for 1000 users. However, at the 99th and 100th percentiles, response times are often higher with our solution, particularly for large loads (500 and 1000 users). This is due to the complexity of update operations and the performance of hardware resources.

B. Discussion

Despite the performance of our solution, certain limitations must be mentioned:

- High response times at 99th and 100th percentiles: For the slowest requests (1% of cases), response times can be significantly higher with our solution, especially under heavy loads. This is due to the complexity of update operations (PATCH) and synchronization between cache and database.
- Failures under high load: Although rare, a few failures were observed under a load of 1000 users (2 failures with cache, 4 failures without cache). These failures are mainly due to timeouts during communication between services.

5. Conclusion

This work validates the effectiveness of our distributed cache solution, while opening up prospects for future research and development. Among the avenues for improvement, we could explore the implementation of an automatic cache node detection mechanism, allowing for dynamic monitoring and removal of failed nodes from the hash circle, while adding new nodes when necessary. This would avoid manual configuration and improve system resilience. In

addition, better POST event management could be integrated to automatically update the cache when new resources are added to the database, ensuring greater consistency between cache and database. These improvements, combined with an increase in hardware resources, could enhance the system's scalability and reliability, enabling it to meet even more demanding loads in real production environment.

6. References

[1]. L. Li, C. Ye, and H. Zhou, "Cache Replacement Algorithm Based on Dynamic Constraints in Microservice Platform," in 2022 *International Conference on Service Science (ICSS)*, (Zhuhai, China), pp. 167-174, IEEE, May 2022.

[2]. Y. Gao, G. Casale, and R. Singhal, "Performance Modeling of Distributed Data Processing in Microservice Applications," *ACM Transactions on Modeling and Performance Evaluation of Computing Systems*, vol. 9, pp. 1-30, Dec. 2024.

[3]. D. Seth, A. Singh, and S. Panyam, "Distributed caching challenges and strategies in enterprise applications," *International Research Journal of Modernization in Engineering Technology and Science*, Apr. 2024.

[4]. D. Faustino, N. Gonçalves, M. Portela, and A. R. Silva, "Stepwise Migration of a Monolith to a Microservices Architecture: Performance and Migration Effort Evaluation," Jan. 2022.

[5]. B. Yan, H. Gao, H. Wu, W. Zhang, L. Hua, and T. Huang, "Hermes: Efficient Cache Management for Container-based Serverless Computing," in *Proceedings of the 12th Asia-Pacific Symposium on Internetware, (Singapore Singapore)*, pp. 136-145, ACM, Nov. 2020.

[6]. H. Shukur, S. Zeebaree, R. Zebari, O. Ahmed, L. Haji, and D. Abdulqader, "Cache Coherence Protocols in Distributed Systems," *Journal of Applied Science and Technology Trends*, vol. 1, pp. 92-97, June 2020.

[7]. R. Laigner, Y. Zhou, M. A. V. Salles, Y. Liu, and M. Kalinowski, "Data Management in Microservices: State of the Practice, Challenges, and Research Directions," Aug. 2021.

[8]. A. Ferragut, I. Rodriguez, and F. Paganini, "Optimizing TTL Caches under Heavy-Tailed Demands," in *Proceedings of the 2016 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Science*, (Antibes Juan-les-Pins France), pp. 101-112, ACM, June 2016.

[9]. J. Liu, X. Wan, Q. Zhu, T. Peng, and X. Hu, "Research on Adaptive Cache Mechanism Based on TTL," in 2022 2nd *International Conference on Networking, Communications and Information Technology (NetCIT), (Manchester, United Kingdom)*, pp. 507-511, IEEE, Dec. 2022.

[10]. V. Sourlas, G. S. Paschos, P. Flegkas, and L. Tassiulas, "Caching in Content-Based Publish/Subscribe Systems," (Honolulu, Hawaii), pp. 1-6, IEEE, 2009.

[11]. Crudu and MoldStud Research Team, "Effective Strategies for Cache Invalidation in Distributed Systems | MoldStud," Feb. 2025. Retrieved February 16, 2025, from https://moldstud.com/articles/p-master-cache-invalidation-in-distributed-systems-effectively.

[12]. J. P. de Araujo, "A communication-efficient causal broadcast publish/subscribe system," 2019.

[13]. The NCache Team, "NCache Performance Benchmarks," Aug. 2019.

[14]. N. A. N. Sobri, M. A. H. Abas, I. M. Yassin, M. Syahirul, A. M. Ali, N. Tahir, A. Zabidi, and Z. I. Rizman, "Analyzing Latency Performance of Different Cache Methods for Microservice Architecture," vol. 71, no. 3, 2022.

[15]. H. Zhang, K. Kallas, S. Pavlatos, R. Alur, S. Angel, and V. Liu, "MuCache: A General Framework for Caching in Microservice Graphs," Apr. 2024.

[16]. Hamdard University, S. I. Alam, S. Wazir, A. Khalique, and S. I. Hassan, "Data Cache with Distributed Cache: A Design Approach," *International Journal of Computer Science and Engineering*, vol. 4, pp. 17-23, June 2017.

[17]. Independent Researcher, California, USA and A. Bhole, "Caching Strategies to Enhance Performance in Microservices," *International Journal of Scientific Research in Engineering and Management*, vol. 08, pp. 1-6, Dec. 2024.
[18]. S. Newman, "Building Microservices," Aug. 2021.
[19]. V. B. Ramu, "Performance Impact of Microservices Architecture," The Review of Contemporary Scientific and Academic Studies, vol. 3, June 2023.
[20]. R. Nishtala, H. Fugal, S. Grimm, M. Kwiatkowski, H. Lee, H. C. Li, R. McElroy, M. Paleczny, D. Peek, P. Saab, D. Stafford, T. Tung, and V. Venkataramani, "Scaling Memcache at Facebook," 2013.
[21]. D. Karger, E. Lehman, T. Leighton, R. Panigrahy, M. Levine, and D. Lewin, "Consistent hashing and random trees: Distributed caching protocols for relieving hot spots on the World Wide Web," in *Proceedings of the Twenty-Ninth Annual ACM Symposium on Theory of Computing - STOC '97, (El Paso, Texas, United States)*, pp. 654-663, ACM Press, 1997.



Abdou Wahidi BELLO holds PhD from the University of Abomey-Calavi, Benin. He is a consultant in the field of computer analysis, Web and mobile developer.



Abdoul Kamal ASSOUMA holds PhD from the University of Abomey-Calavi, Benin. His research focuses on: biometrics, signal processing and images, computer intelligence, industrial applications and symbolic programming. He is a consultant in the field of computer analysis, web and mobile developer.



Tahirou Djara is a Full Professor of CAMES universities and a lecturer-researcher at the Polytechnic School of Abomey-Calavi in University of Abomey-Calavi, Benin. His research interests include biometrics, signal and image processing, computational intelligence, industrial applications, and symbolic programming. He received his PhD in signal and image processing from the University of Abomey-Calavi in 2013. He is a consultant in quality assurance in higher education and a consultant in engineering science and technology.



Emmanuel Godwin HOUENOU is a design engineer who graduated from the University of Abomey-Calavi. A full-stack developer, he excels at creating engaging front-end experiences with React, while integrating robust back-end functionality with Django. He specializes in creating user-friendly, efficient, and maintainable web solutions tailored to a variety of project needs.